

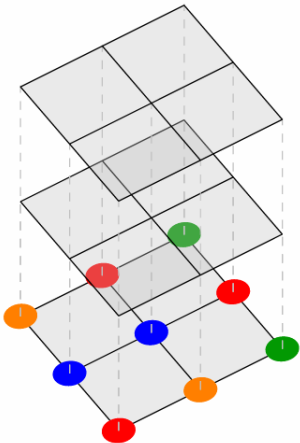
OpenROAD

Global Routing

Edgar Perner

Hands On Tutorial for Physical Design with Open Source Tooling
21–23 September

Routing



Pin: connection point on a cell.

Net: Set of pins (of the same color). Nets must become electrically connected.

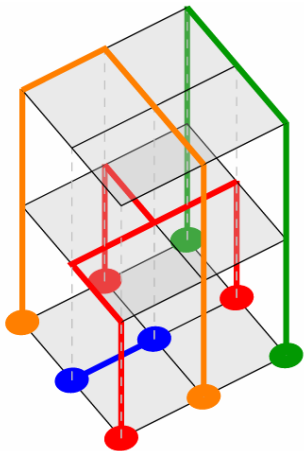
Stacked planes: metal **layers**.

Horizontal edges: **wire** segments.

Vertical edges: **vias**.

Routing task: choose a route for every net.

Routing



Pin: connection point on a cell.

Net: Set of pins (of the same color). Nets must become electrically connected.

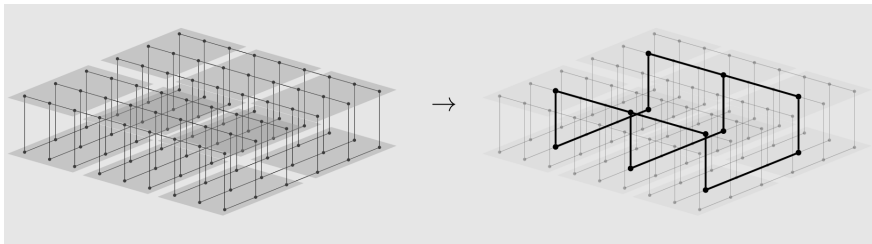
Stacked planes: metal **layers**.

Horizontal edges: **wire** segments.

Vertical edges: **vias**.

Routing task: choose a route for every net.

The Global Routing simplification



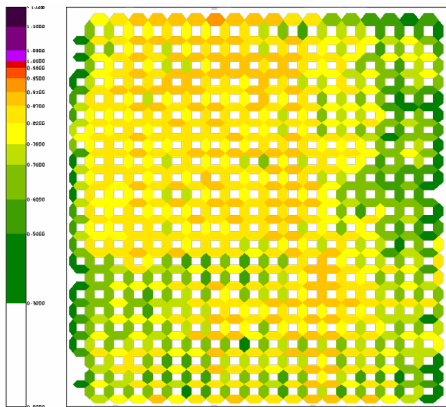
Replace detailed graph with a capacity graph

Drop local technology constraints

Different nets may share vertices and edges.

Capacity: number of free tracks crossing the boundary between two tiles.

Congestion



congestion plot

Regions connected by global edges:

GR tiles or **GCells**

usage := #nets using a global routing edge

congestion := usage / capacity

overflow on edges with congestion > target:

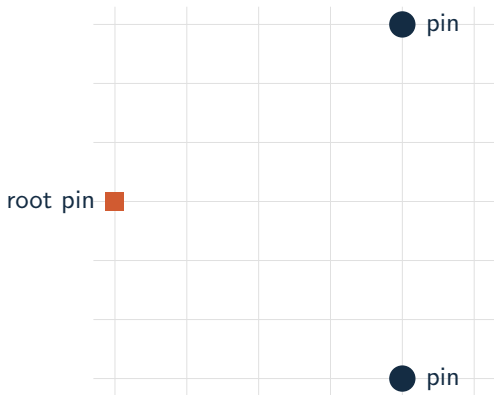
Why do global route?

Global routing assigns each net broad regions and layers. Used as guides by detailed router:

- avoid global dead ends and congestion
- meet timing and power constraints
- search a much smaller space

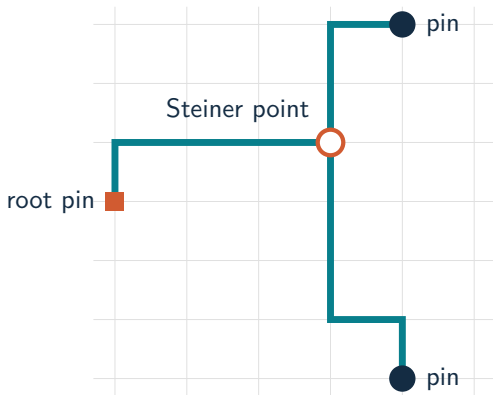
Detailed routing must obey all manufacturing rules.

Rectilinear Steiner trees.



Connect all pins in 2D

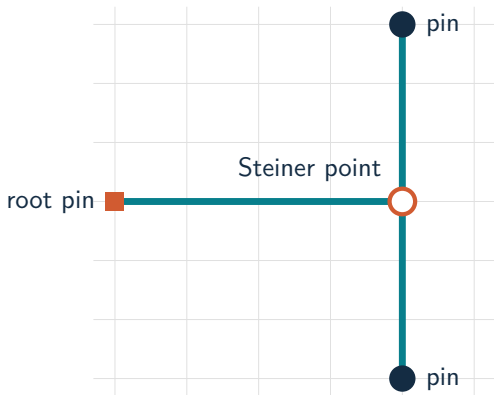
Rectilinear Steiner trees.



Connect all pins in 2D

Tree uses only horizontal and vertical segments

Rectilinear Steiner trees.

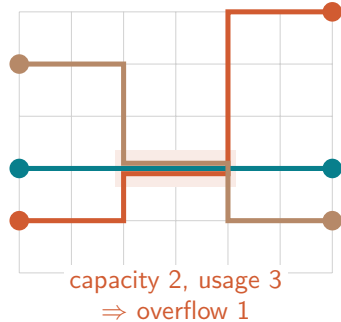


Connect all pins in 2D with minimal wirelength (FLUTE algorithm)

Tree uses only horizontal and vertical segments

Steiner tree packing problem

All trees compete for the same edge capacities.



Approaches

Openroad uses FastRoute:

1. Compute 2D Steiner trees
2. Route + Ripup and reroute in 2D
3. Assign 3D layers

Approaches

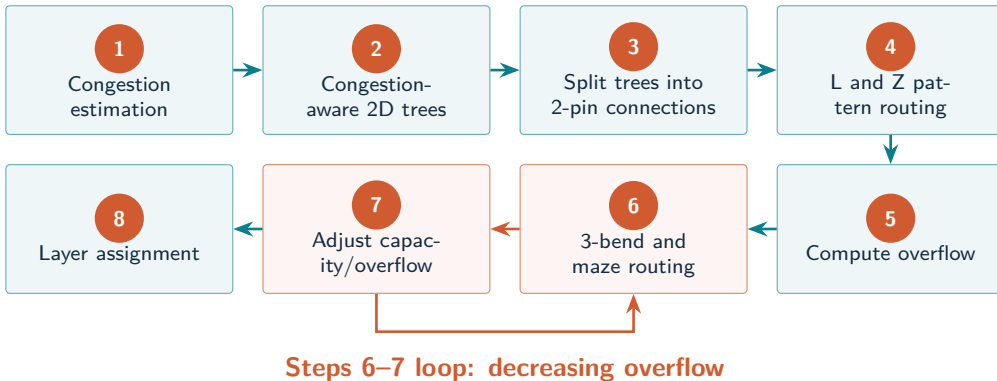
Openroad uses FastRoute:

1. Compute 2D Steiner trees
2. Route + Ripup and reroute in 2D
3. Assign 3D layers

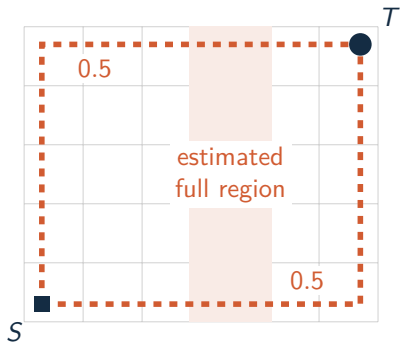
BonnRouteGlobal

1. Compute 3D Steiner trees
2. Recompute trees based on current and past congestion
3. Choose packing of trees

The FastRoute steps



1 Congestion estimation

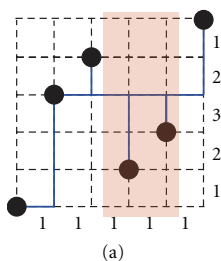


Two L candidates: 0.5 demand each

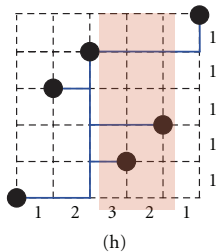
1. FLUTE trees
2. Tree edges $\rightarrow$ 2-pin connections
3. Initially two L routes, 0.5 each
4. Iteratively decide on better L-shape

Output: congestion map for step 2

2 Congestion-aware 2D trees

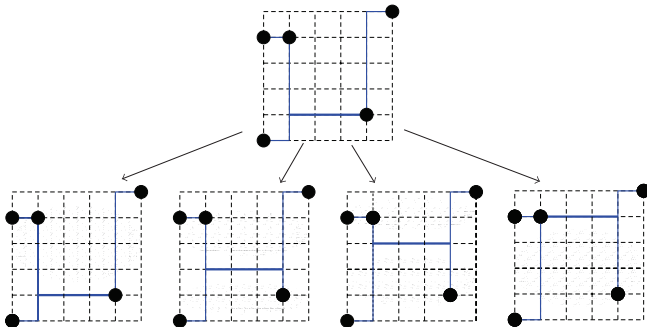


High horizontal congestion



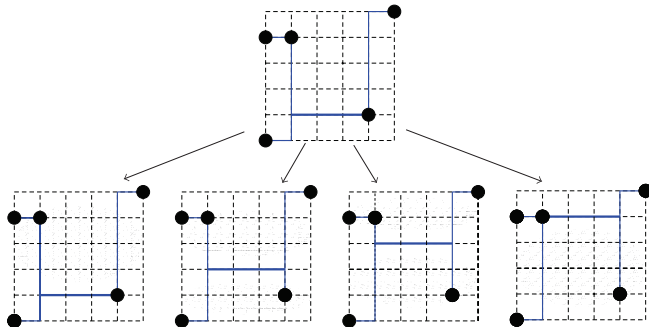
Input: congestion map
Edge congestion by row / column
⇓
Stretch overflow-cut distances
⇓
Run FLUTE again

2 Segment shifting



Movable Steiner segments
Lower congestion,
unchanged wirelength

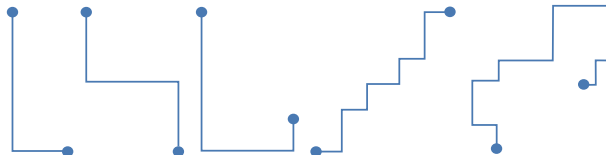
2 Segment shifting



Movable Steiner segments
Lower congestion,
unchanged wirelength

3 Divide Tree into 2-pin connections

4 L and Z pattern initialization

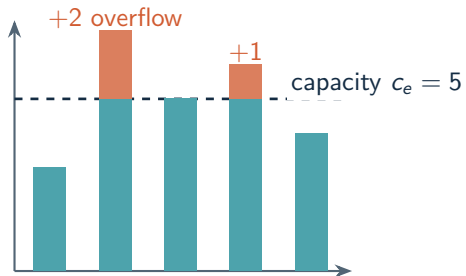


Route classes: L, Z, U, monotonic, maze

Choose best **L** or **Z** shape in 2D global routing graph for all 2 pin connections

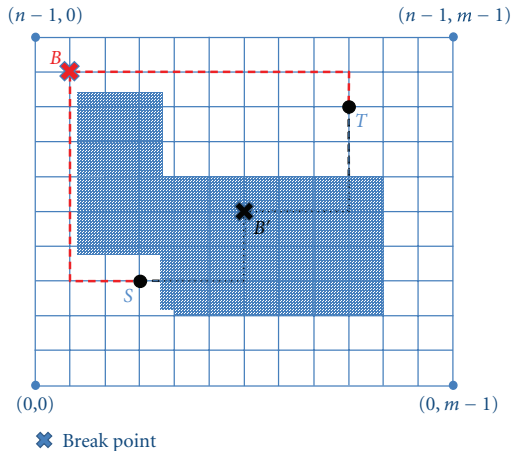
Other patterns: Only in later stages

5 Capacity/overflow computation



Set edge prices:
Edgelenlength + Overflow penalty

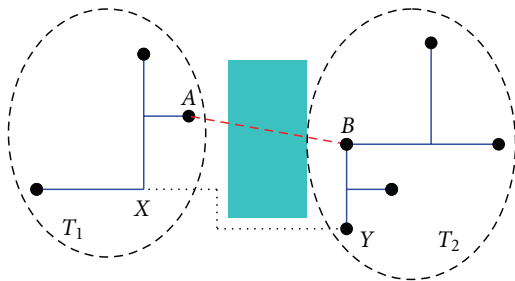
6 3-bend routing



L / Z: both regions blocked

Choose cheapest 3-bend: two L pieces
+ break point B

6 Maze routing



Cut $A-B$: subtrees

Dijkstra: $X-Y$ is least-cost reconnection of T_1 , T_2

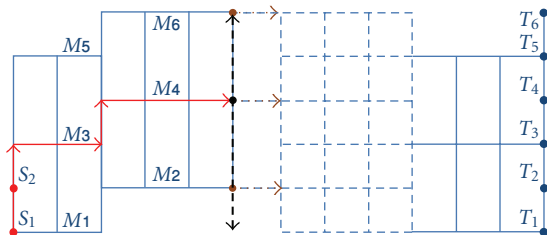
7 Overflow price feedback

Update overflows and prices at all changed edges. Increase overflow penalty weight.



Loop: steps 6–7 while total overflow falls

8 Spiral layer assignment



Assign $S-T$ path

Horizontal: step on 2D path

Vertical: layer change

Order

Start with nets that are short or have many pins

Within nets: start near pins, spiral inward

Dynamic program for one path

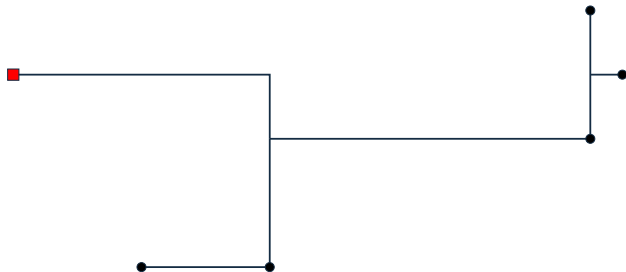
Compute $D(i, \ell)$: fewest vias through segment i on layer ℓ

Prim-Dijkstra: FLUTE alternative for timing optimization

Build tree T iteratively by adding connection from u to a new pin v minimizing:

$$\underbrace{d(u, v)}_{\text{new connection length}} + \alpha \underbrace{d_T(r, u)}_{\text{path to root}}$$

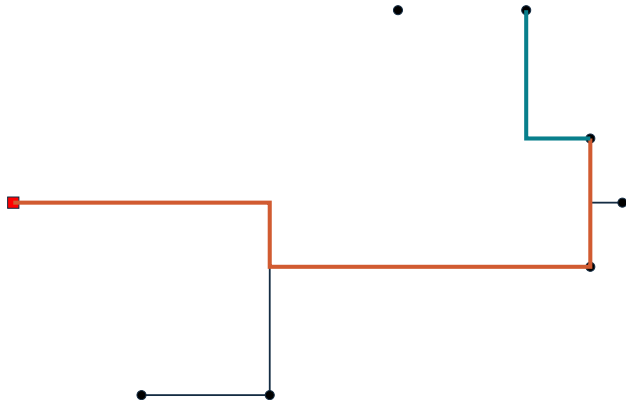
• •



Prim-Dijkstra: FLUTE alternative for timing optimization

Build tree T iteratively by adding connection from u to a new pin v minimizing:

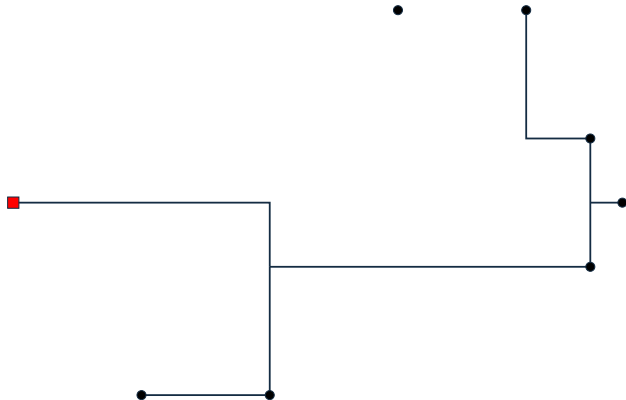
$$\underbrace{d(u, v)}_{\text{new connection length}} + \alpha \underbrace{d_T(r, u)}_{\text{path to root}}$$



Prim-Dijkstra: FLUTE alternative for timing optimization

Build tree T iteratively by adding connection from u to a new pin v minimizing:

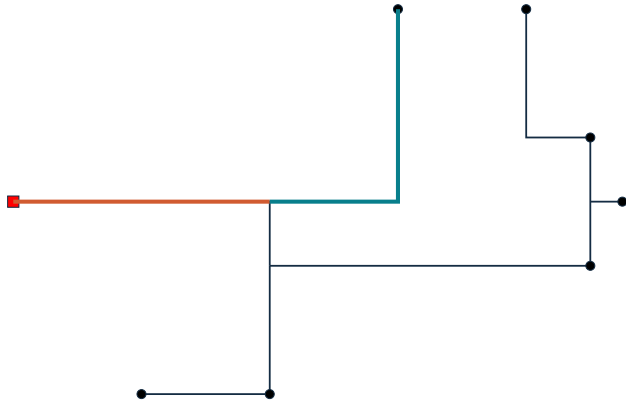
$$\underbrace{d(u, v)}_{\text{new connection length}} + \alpha \underbrace{d_T(r, u)}_{\text{path to root}}$$



Prim-Dijkstra: FLUTE alternative for timing optimization

Build tree T iteratively by adding connection from u to a new pin v minimizing:

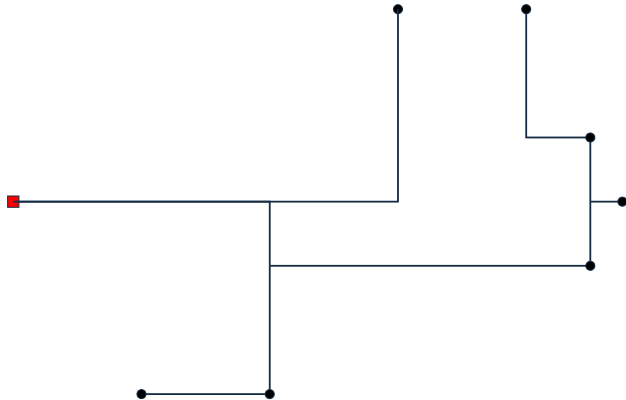
$$\underbrace{d(u, v)}_{\text{new connection length}} + \alpha \underbrace{d_T(r, u)}_{\text{path to root}}$$



Prim-Dijkstra: FLUTE alternative for timing optimization

Build tree T iteratively by adding connection from u to a new pin v minimizing:

$$\underbrace{d(u, v)}_{\text{new connection length}} + \alpha \underbrace{d_T(r, u)}_{\text{path to root}}$$



Prim-Dijkstra: FLUTE alternative for timing optimization

$$\underbrace{d(u, v)}_{\text{new connection length}} + \alpha \underbrace{d_T(r, u)}_{\text{path to root}}$$

$\alpha = 0$: **Prim**

Seek low total tree length.

$0 < \alpha < 1$

Balance added length
against path depth.

$\alpha = 1$: **Dijkstra**

Minimize longest path to
root.

```
set_routing_alpha -net critical_net 0.5
```

OpenROAD Tcl commands

```
set_routing_layers -signal Metal2-Metal6
```

```
# Reduce capacity on all layers by 30%  
set_global_routing_layer_adjustment * 0.3
```

```
# Use FLUTE (default)  
set_routing_alpha 0.0
```

```
global_route -guide_file route.guide \  
    -congestion_iterations 50 \  
    #percentage of most timing critical nets  
    -critical_nets_percentage 10
```

Appendix: Checking

```
report_wire_length -global_route \  
    -summary  
  
# GUI: replace with an existing net.  
draw_route_segments {example_net} \  
    -show_pin_locations  
  
# With libraries, SDC and wire RC set:  
estimate_parasitics -global_routing  
report_checks -path_delay max
```

Appendix: Seeing the intermediate trees

```
# In an OpenROAD GUI session, before global_route:  
global_route_debug -net example_net \  
    -st -rst -tree2D -tree3D
```

```
global_route -guide_file route.guide
```

- st Tree produced by the Steiner-tree builder.
- rst Rectilinear tree in the global router.
- tree2D Tree after congestion iterations.
- tree3D Tree after metal-layer assignment.

Appendix: Integration with the rest of OpenROAD

```
# Reroute only nets affected by supported design edits:
```

```
global_route -start_incremental
```

```
# ... perform the design edits here ...
```

```
global_route -end_incremental
```

```
# Optional process-antenna repair, with technology support:
```

```
repair_antennas
```

Antenna violations: manufacturing charge can damage gates. Diodes can provide protection.

Antenna repair needs technology rules and diode cells. Run before filler placement.